МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Львівський національний університет імені Івана Франка

Факультет електроніки та комп’ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ № 6

з дисципліни “Професійний Python”

на тему “ Автоматизоване тестування Python-програм “

Виконав

студент 3 курсу

групи ФЕП-32

Козачок Ростислав Михайлович

Перевірив

Куньо Іван Михайлович

Львів – 2026

Мета: Сформувати практичні навички професійного автоматизованого тестування Python-програм із використанням фреймворку pytest, навчитися проєктувати unit та integration tests, створювати fixtures, використовувати параметризацію, перевіряти exceptions, ізолювати файлову систему, застосовувати mocks і аналізувати test coverage.

Варіант: 14 (Система керування завданнями)

Хід роботи:

1. Опис тестованого проєкту

Проєкт реалізує систему керування завданнями (Task Management System). Основні сутності — Task, BugTask, FeatureTask, які зберігаються та обробляються в TaskBoard. Реалізовано валідацію статусів і пріоритетів, бізнес-логіку зміни станів (наприклад, заборона відкриття закритого завдання), а також імпорт та експорт даних у форматах CSV та JSON.

2. Структура проєкту

Production code:

- src/tasks/core.py — містить моделі (Task, BugTask, FeatureTask), сервіс TaskBoard для управління колекцією завдань, TaskWorkflowService для взаємодії із зовнішніми залежностями, та функції імпорту/експорту.

Test suite:

- tests/conftest.py — містить спільні фікстури (basic_task, bug_task, populated_board).
- tests/test_tasks.py — містить набір unit та integration тестів (20+ тестів), розділених за логічними блоками (моделі, board, mocks, monkeypatch, files, integration).

3. Test Matrix (Матриця тестування)

Категорія	Приклад у проєкті
Normal cases	Зміна статусу на "In Progress", додавання валідного завдання на дошку.
Boundary cases	Розрахунок середнього часу (0 годин, якщо дошка порожня).
Invalid data	Створення завдання з негативним часом (estimated_hours = -2.0).

Категорія	Приклад у проєкті
State transition	Спроба перевести статус із Closed у In Progress (помилка).
Missing data	Імпорт CSV-файлу без обов'язкового поля title.
Dependency failure	Помилка збереження в базу даних або падіння сервісу сповіщень (перевірка side_effect).

4. Приклади реалізації вимог

Приклад фікстури (conftest.py):

```
@pytest.fixture
def populated_board(basic_task, bug_task):
    board = TaskBoard()
    board.add_task(basic_task)
    board.add_task(bug_task)
    return board
```

Приклад Parametrization:

```
@pytest.mark.parametrize("status", ["In Progress", "Resolved", "Closed"])
def test_valid_status_change(basic_task, status):
    basic_task.change_status(status)
    assert basic_task.status == status
```

Приклад pytest.raises (перевірка Exception та Boundary/Error testing):

```
def test_invalid_status_transition(basic_task):
    basic_task.change_status("Closed")
    with pytest.raises(InvalidStatusTransition, match="Cannot reopen"):
        basic_task.change_status("In Progress")
```

Приклад pytest.approx (порівняння float):

```
def test_average_estimation_approx(populated_board):
    # 4.5 + 1.5 = 6.0 / 2 = 3.0
    assert populated_board.average_estimation() == pytest.approx(3.0)
```

Приклад Mock (return_value, interaction assertion) та side_effect:

```
def test_workflow_complete_task_success(basic_task):
    mock_repo = Mock()
    mock_repo.save.return_value = True  # return_value
    mock_notifier = Mock()
```

```
    service = TaskWorkflowService(mock_repo, mock_notifier)
    service.complete_task(basic_task)

    mock_repo.save.assert_called_once_with(basic_task) # interaction assertion

def test_workflow_notifier_side_effect(basic_task):
    mock_repo, mock_notifier = Mock(), Mock()
    mock_repo.save.return_value = True
    mock_notifier.send_alert.side_effect = ConnectionError("Network down") # side_effect

    service = TaskWorkflowService(mock_repo, mock_notifier)
    with pytest.raises(ConnectionError):
        service.complete_task(basic_task)
```

Приклад AsyncMock:

```
@pytest.mark.asyncio
async def test_async_fetch_remote_task():
    async_repo = AsyncMock(spec=AsyncTaskRepository)
    async_repo.fetch_remote_task.return_value = {"id": 99, "status": "Remote"}

    result = await async_repo.fetch_remote_task(99)
    async_repo.fetch_remote_task.assert_awaited_once_with(99)
```

Приклад tmp_path (ізоляція файлової системи):

```
@pytest.mark.integration
def test_export_json_tmp_path(tmp_path, populated_board):
    file_path = tmp_path / "tasks_export.json"
    export_tasks_json(populated_board._tasks, file_path)
    assert file_path.exists()
```

Приклад monkeypatch (підміна конфігурації):

```
def test_config_allowed_statuses(monkeypatch):
    monkeypatch.setenv("ALLOWED_STATUSES", "Draft,Published")
    assert get_allowed_statuses() == ["Draft", "Published"]
```

Приклад Integration Test:

```
@pytest.mark.integration
def test_full_application_flow(tmp_path):
    # Імпорт -> Додавання на дошку -> Обробка сервісом (Workflow) -> Експорт
    csv_file = tmp_path / "data.csv"
    csv_file.write_text("task_id,title,priority,estimated_hours\n1,Write tests,High,3.0\n", encoding="utf-8")
    tasks = import_tasks_csv(csv_file)

    board = TaskBoard()
    board.add_task(tasks[0])
```

```
mock_repo = Mock()
mock_repo.save.return_value = True
service = TaskWorkflowService(mock_repo, Mock())
service.complete_task(board.find_task(1))

json_out = tmp_path / "out.json"
export_tasks_json(board._tasks, json_out)

result_data = json.loads(json_out.read_text(encoding="utf-8"))
assert result_data[0]["status"] == "Closed"
```

5. Результати виконання та покриття коду (Coverage)

Загальна кількість тестів: **21** (всі passed).

Звіт pytest-cov (термінал):

```
Name                Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------
src/tasks/core.py      67      0     18      0   100%
-----------------------------------------------------------------
TOTAL                  67      0     18      0   100%
```

Аналіз недостатньо протестованих шляхів: Недостатньо протестованих шляхів не виявлено. Statement та Branch coverage становлять 100%. Усі гілки умов (if/else), включаючи перевірку помилок та некоректних даних при імпорті CSV, повністю покриті тестами.

Опис знайдених дефектів: Під час розробки тестів виявлено потенційний дефект у логіці валідації пріоритетів: при ініціалізації класу Task не перевірялося значення пріоритету, що дозволяло створювати завдання з неіснуючим пріоритетом (наприклад, "SuperHigh"). У core.py було додано __post_init__ для строгої валідації полів, що повністю усунуло проблему (перевіряється в test_invalid_priority_raises).

Висновок:

Під час виконання лабораторної роботи було створено повноцінний набір автоматизованих тестів для системи керування завданнями з використанням фреймворку pytest. Застосування фікстур та параметризації дозволило зменшити дублювання коду в тестах. Використання об'єктів Mock та AsyncMock дало змогу повністю ізолювати бізнес-логіку від зовнішніх сервісів та симулювати сценарії відмов (через side_effect). Робота з файловою системою була надійно ізольована завдяки tmp_path. В результаті було досягнуто 100% покриття коду за гілками та інструкціями (branch and statement coverage).